

1일차 : GUI와 파일 입출력

2026.07

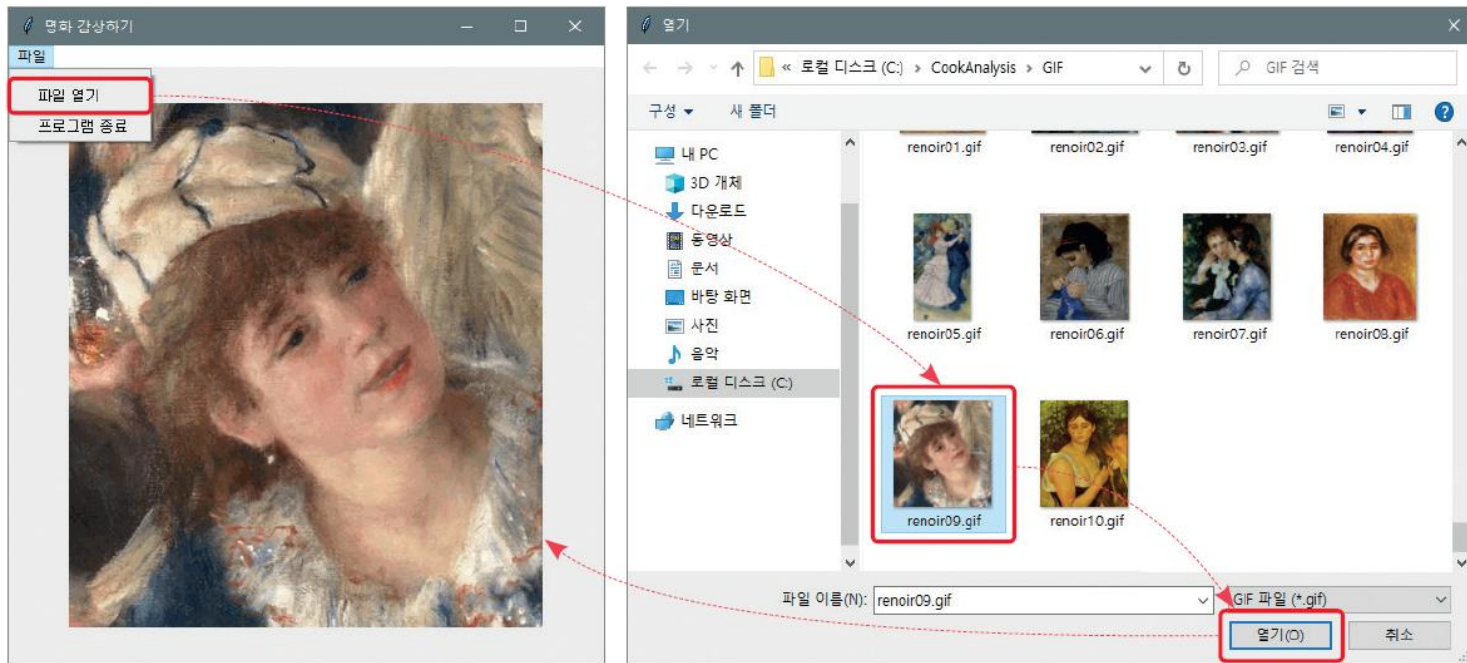
학습 목표

- 윈도 프로그래밍의 개념을 이해할 수 있다.
- 윈도 프로그래밍에서 활용되는 다양한 위젯을 이해할 수 있다.
- 메뉴의 구성을 이용한 고급 윈도 프로그래밍을 이해할 수 있다.
- 파일 처리 개념을 이해할 수 있다.
- 텍스트 파일의 내용을 읽거나 쓰는 기능을 이해할 수 있다.
- 이진 파일 중 사진 파일을 윈도창에 출력하는 방법을 이해할 수 있다.

01 실습 프로그램

■ [프로그램 1] 영화 감상

- 메뉴를 활용하는 영화 감상 프로그램



01 실습 프로그램

■ [프로그램 2] 흑백 사진 출력

- 사진 파일을 읽어서 출력하는 프로그램
- 이진 파일 처리가 가능하도록 일반적인 GIF, JPG, BMP 등과는 다른 특별한 용도의 흑백 파일을 사용할 예정



02 기본 위젯

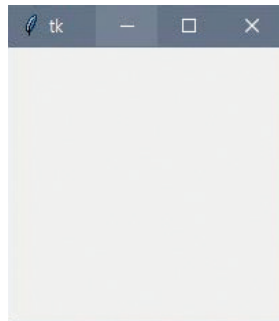
■ 윈도우창

- 기본적인 윈도우창의 구성
- 가장 간단한 윈도우창을 표시하는 코드

Code01-01.py

```
01  from tkinter import *
02
03  window = Tk()
04
05  ## 이 부분에서 화면을 구성하고 처리함 ##
06
07  window.mainloop()
```

제목 표시



※tkinter는 파이썬에서 GUI(Graphical User Interface) 프로그래밍을 할 때 사용하는 모듈임.
(GUI는 사용자가 그래픽을 통하여 컴퓨터와 상호작용할 수 있는 환경을 의미함)

02 기본 위젯

■ 윈도우창

- 크기를 고정할 수도 있음

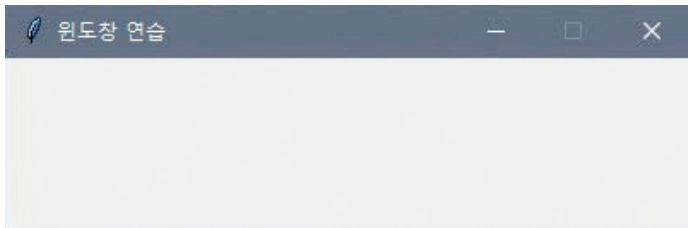
Code01-02.py

```
01 from tkinter import *  
02  
03 window = Tk()  
04 window.title("윈도창 연습")  
05 window.geometry("400x100")  
06 window.resizable(width = FALSE, height = FALSE)  
07  
08 window.mainloop()
```

제목 표시

크기 지정

크기 고정



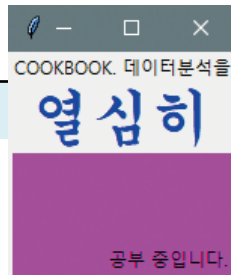
02 기본 위젯

■ 레이블

- 문자를 표현할 수 있는 위젯
- 'Label(부모윈도, 옵션...)' 형식으로 사용하고 '옵션'으로 위젯을 다양하게 설정
- pack() 함수를 호출해야 화면에 표시됨

Code01-03.py

```
01  from tkinter import *
02  window = Tk()
03
04  label1 = Label(window, text = "COOKBOOK, 데이터분석을")
05  label2 = Label(window, text = "열심히", font = ("궁서체", 30), fg = "blue")
06  label3 = Label(window, text = "공부 중입니다.", bg = "magenta", width = 20, height = 5, anchor = SE)
07
08  label1.pack()
09  label2.pack()
10  label3.pack()
11
12  window.mainloop()
```



02 기본 위젯

■ 레이블

표 5-1 옵션의 종류

옵션	설명
text	글자 자체를 나타낸다.
font	글꼴과 크기를 지정한다.
fg	foreground의 약어로 글자색을 지정한다.
bg	background의 약어로 배경색을 지정한다.
width	위젯의 폭을 지정한다.
anchor	위젯이 어느 위치에 자리 잡을지 결정한다. South East의 약어이다. anchor에는 N, NE, E, SE, S, SW, W, NW, CENTER 등의 값을 사용할 수 있으며, 기본값은 CENTER이다.

02 기본 위젯

■ 레이블

- 레이블에는 글자 대신 이미지를 넣을 수도 있음
- PhotoImage()는 GIF 파일만 지원하고 JPEG나 BMP 등은 지원하지 않으니 주의

Code01-04.py

```
01 from tkinter import *  
02 window = Tk()  
03  
04 photo = PhotoImage(file = "gif/dog.gif")  
05 label1 = Label(window, image = photo)  
06  
07 label1.pack()  
08  
09 window.mainloop()
```



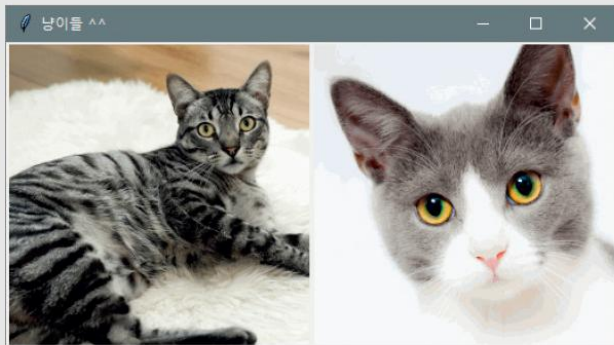
02 기본 위젯

■ 레이블

SELF STUDY 5-1

Code05-04.py를 수정해서 이미지 2개를 출력해 보자.

힌트 위젯을 가로로 나타내려면 `pack(side=LEFT)`를 사용한다.



02 기본 위젯

■ 버튼

- 마우스로 클릭하면 놀리는 효과와 함께 지정한 작업이 실행되도록 하는 위젯
 - 'Button(부모윈도, 옵션...)' 형식으로 사용
 - 레이블과 달리 버튼은 눌렀을 때 command 옵션을 통해 특정 작업을 하도록 직접 설정
-
- 버튼을 누르면 파이썬 IDLE이 종료되는 코드

Code01-05.py

```
01 from tkinter import *
02 window = Tk()
03
04 button1 = Button(window, text = "파이썬 종료", fg = "red", command = quit )
05
06 button1.pack()
07
08 window.mainloop()
```



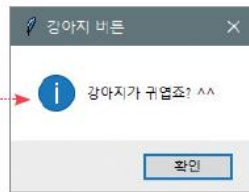
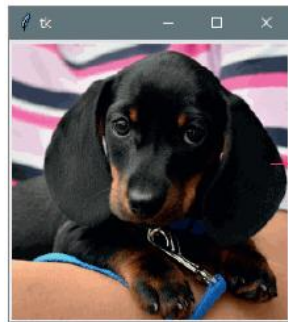
02 기본 위젯

■ 버튼

- 이미지 버튼을 누르면 간단한 메시지창이 나오는 코드

Code01-06.py

```
01 from tkinter import *
02 from tkinter import messagebox
03
04 ## 함수 선언 부분 ##
05 def myFunc() :
06     messagebox.showinfo("강아지 버튼", "강아지가 귀엽죠? ^^")
07
08 ## 메인 코드 부분 ##
09 window = Tk()
10
11 photo = PhotoImage(file = "gif/dog2.gif")
12 button1 = Button(window, image = photo, command = myFunc)
13
14 button1.pack()
15
16 window.mainloop()
```



02 기본 위젯

■ 체크버튼

- 체크버튼(Checkbutton)은 켜고 끄는 데 사용하는 위젯
- 'Checkbutton(부모윈도, 옵션...)' 형식으로 사용

02 기본 위젯

■ 체크버튼

- 체크버튼을 켜거나 끄면 메시지창이 열리게 하는 코드

Code01-07.py

```
01  from tkinter import *
02  from tkinter import messagebox
03  window = Tk()
04
05  ## 함수 선언 부분 ##
06  def myFunc() :
07      if chk.get() == 0 :
08          messagebox.showinfo("", "체크버튼이 꺼졌어요.")
09      else :
10          messagebox.showinfo("", "체크버튼이 켜졌어요.")
11
12  ## 메인 코드 부분 ##
13  chk = IntVar()
14  cb1 = Checkbutton(window, text = "클릭하세요", variable = chk, command = myFunc)
15
16  cb1.pack()
17
18  window.mainloop()
```



02 기본 위젯

■ 라디오버튼

- 라디오버튼(Radiobutton)은 여러 선택지 중 하나를 선택할 때 사용하는 위젯
- 'Radio button(부모윈도, 옵션...)' 형식으로 사용

Code01-08.py

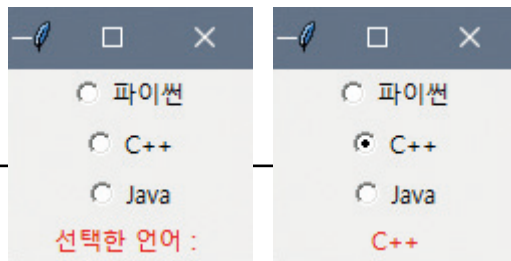
```
01  from tkinter import *
02  window = Tk()
03
04  ## 함수 선언 부분 ##
05  def myFunc() :
06      if var.get() == 1 :
07          label1.configure(text = "파이썬")
08      elif var.get() == 2 :
09          label1.configure(text = "C++")
10      else :
11          label1.configure(text = "Java")
12
13  ## 메인 코드 부분 ##
14  var = IntVar()
15  rb1 = Radiobutton(window, text = "파이썬", variable = var, value = 1 , command = myFunc)
```

02 기본 위젯

■ 라디오버튼

Code01-08.py

```
16 rb2 = Radiobutton(window, text = "C++", variable = var, value = 2 , command = myFunc)
17 rb3 = Radiobutton(window, text = "Java", variable = var, value = 3 , command = myFunc)
18
19 label1 = Label(window, text = "선택한 언어 :", fg = "red")
20
21 rb1.pack()
22 rb2.pack()
23 rb3.pack()
24 label1.pack()
25
26 window.mainloop()
```



03 위젯 설정

■ 위젯 배치

- 윈도우창에 위젯을 여러 개 표시할 때는 그 배치를 잘 고려해야 효과적으로 화면 표현 가능
- `pack()`이나 `place()` 함수를 사용해 위젯을 화면에 출력
- 기존에 출력된 위젯의 모양을 변경할 때는 `configure()` 함수를 사용

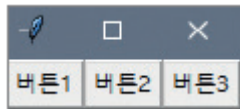
03 위젯 설정

■ 수평 정렬

- pack() 함수의 옵션 중에서 수평으로 정렬하는 방법은 side=LEFT 혹은 side=RIGHT

Code01-09.py

```
01 from tkinter import *
02 window = Tk()
03
04 button1 = Button(window, text = "버튼1")
05 button2 = Button(window, text = "버튼2")
06 button3 = Button(window, text = "버튼3")
07
08 button1.pack(side = LEFT)
09 button2.pack(side = LEFT)
10 button3.pack(side = LEFT)
11
12 window.mainloop()
```



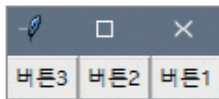
03 위젯 설정

■ 수평 정렬

- for 문을 이용하여 오른쪽부터 채우는 코드

Code01-10.py

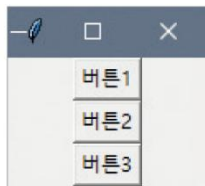
```
01  from tkinter import *
02  window = Tk()
03
04  btnList = [None] * 3
05
06  for i in range(0, 3) :
07      btnList[i] = Button(window, text = "버튼" + str(i + 1))
08
09  for btn in btnList :
10      btn.pack(side = RIGHT)
11
12  window.mainloop()
```



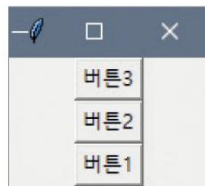
03 위젯 설정

■ 수직 정렬

- pack() 함수의 옵션 중 수직으로 정렬하는 방법은 side=TOP 혹은 side=BOTTOM



```
btn.pack(side = TOP)
```



```
btn.pack(side = BOTTOM)
```

03 위젯 설정

■ 폭 조정

- pack() 함수의 옵션 중 윈도우창 폭에 맞추는 방법은 fill=X



```
btn.pack(side = TOP, fill = X)
```

- pack() 함수의 옵션 중 위젯 사이에 여백을 주는 방법은 padx=픽셀값 또는 pady=픽셀값



```
btn.pack(side = TOP, fill = X, padx = 10, pady = 10)
```

03 위젯 설정

■ 여백 조절

- pack() 함수의 옵션 중 위젯 내부에 여백을 주는 방법은 ipadx=픽셀값 또는 ipady=픽셀값



```
btn.pack(side = TOP, fill = X, ipadx = 10, ipady = 10)
```

- 필요하다면 위젯 내부와 외부에 모두 여백도 줄 수 있음



```
btn.pack(side = TOP, fill = X, ipadx = 10, ipady = 10, padx = 10, pady = 10)
```

04 메뉴와 대화상자

■ 메뉴 생성

■ 메뉴의 구성과 형식

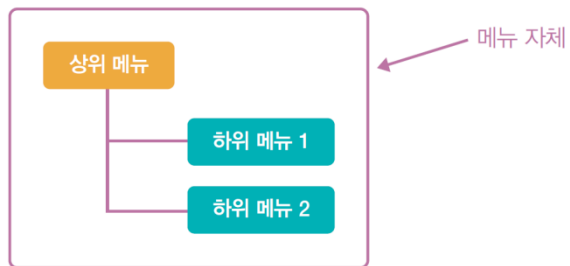


그림 5-1 메뉴의 구성도

```
메뉴자체 = Menu(부모윈도)  
부모윈도.config(menu = 메뉴자체)
```

```
상위메뉴 = Menu(메뉴자체)  
메뉴자체.add_cascade(label = "상위메뉴텍스트", menu = 상위메뉴)  
상위메뉴.add_command(label = "하위메뉴1", command = 함수1)  
상위메뉴.add_command(label = "하위메뉴2", command = 함수2)
```

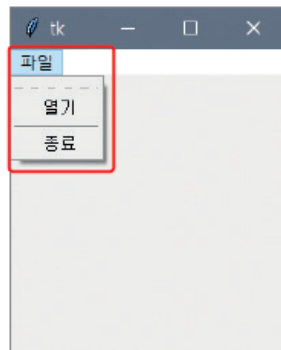
04 메뉴와 대화상자

■ 메뉴 생성

■ 메뉴의 구성과 형식

Code01-11.py

```
01  from tkinter import *
02
03  window = Tk()
04
05  mainMenu = Menu(window)
06  window.config(menu = mainMenu)
07
08  fileMenu = Menu(mainMenu)
09  mainMenu.add_cascade(label = "파일", menu = fileMenu)
10  fileMenu.add_command(label = "열기")
11  fileMenu.add_separator()
12  fileMenu.add_command(label = "종료")
13
14  window.mainloop()
```



04 메뉴와 대화상자

■ 메뉴 생성

- 메뉴를 선택하면 특정 작업이 작동할 수 있도록 추가한 코드

Code01-12.py

```
01  from tkinter import *
02  from tkinter import messagebox
03
04  ## 함수 선언 부분 ##
05  def func_open() :
06      messagebox.showinfo("메뉴선택", "열기 메뉴를 선택함")
07
08  def func_exit() :
09      window.quit()
10      window.destroy()
11
12  ## 메인 코드 부분 ##
13  window = Tk()
14
15  mainMenu = Menu(window)
```

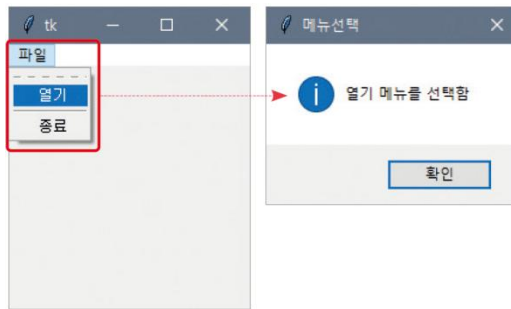
04 메뉴와 대화상자

■ 메뉴 생성

- 메뉴를 선택하면 특정 작업이 작동할 수 있도록 추가한 코드.

Code01-12.py

```
16 window.config(menu = mainMenu)
17
18 fileMenu = Menu(mainMenu)
19 mainMenu.add_cascade(label = "파일", menu = fileMenu)
20 fileMenu.add_command(label = "열기", command =
21 func_open)
22 fileMenu.add_separator()
23 fileMenu.add_command(label = "종료", command = func_exit)
24
    window.mainloop()
```



04 메뉴와 대화상자

■ 대화 상자

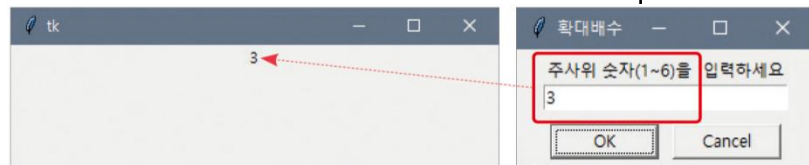
- 파이썬은 몇 가지 대화상자를 제공
- 기본적인 메시지창을 표시 : `messagebox.showinfo()`
- 숫자나 문자를 입력받음 : `tkinter.simpledialog` 모듈을 임포트한 후 `askinteger()`, `askstring()` 함수 등을 사용

04 메뉴와 대화상자

■ 대화 상자

Code01-13.py

```
01  from tkinter import *
02  from tkinter.simpledialog import *
03
04  ## 함수 선언 부분 ##
05  window = Tk()
06  window.geometry("400x100")
07
08  label1 = Label(window, text = "입력된 값")
09  label1.pack()
10
11  value = askinteger("확대배수", "주사위 숫자(1~6)을 입력하세요", minvalue = 1, maxvalue = 6)
12
13  label1.configure(text = str(value))
14  window.mainloop()
```



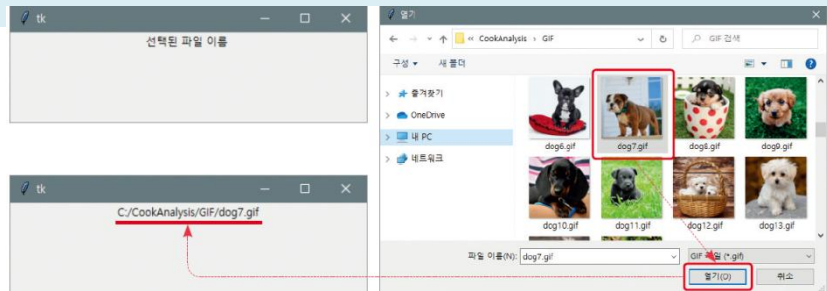
04 메뉴와 대화상자

■ 대화 상자

- 파일을 열거나 저장할 때 표시하는 대화상자: tkinter.filedialog 모듈을 임포트한 후 askopenfilename()이나 asksaveasfile() 함수를 사용

Code01-14.py

```
01 from tkinter import *
02 from tkinter.filedialog import *
03
04 ## 함수 선언 부분 ##
05 window = Tk()
06 window.geometry("400x100")
07
08 label1 = Label(window, text = "선택된 파일 이름")
09 label1.pack()
10
11 filename = askopenfilename(parent = window, filetypes = (("GIF 파일", "*.gif"), ("모든 파일", "*.*")))
12
13 label1.configure(text = str(filename))
14 window.mainloop()
```



04 메뉴와 대화상자

■ 대화 상자

- 파일을 저장하는 대화상자는 매개변수가 약간 다름

Code01-15.py

```
01 from tkinter import *
02 from tkinter.filedialog import *
03
04 ## 함수 선언 부분 ##
05 window = Tk()
06 window.geometry("400x100")
07
08 label1 = Label(window, text = "선택된 파일 이름")
09 label1.pack()
10
11 saveFp = asksaveasfile(parent = window, mode = "w", defaultextension = ".jpg",
12                         filetypes = (("JPG 파일", "*.jpg;*.jpeg"), ("모든 파일", "*.*")))
13 label1.configure(text = saveFp)
14 saveFp.close()
```

04 메뉴와 대화상자

■ 대화 상자

- 파일을 저장하는 대화상자는 매개변수가 약간 다름



04 메뉴와 대화상자

■ 대화 상자

- 주의할 점은 askopenfilename() 함수는 파일 경로와 파일명 문자열만 반환하지만, asksaveasfile() 함수는 다른 정보도 함께 반환한다는 것
- 반환된 값 중에서 name과 관련된 것이 새로 저장할 파일명이 들어 있는 부분

04 메뉴와 대화상자

■ [프로그램 1] 완성

- 영화 감상 프로그램의 핵심은 메뉴 처리와 파일 처리

Code01-16.py

```
01  from tkinter import *
02  from tkinter.filedialog import *
03
04  ## 함수 선언 부분 ##
05  def func_open() :
06      filename = askopenfilename(parent = window, filetypes = (("GIF 파일", "*.gif"), "모든 파일", "*..*"))
07      photo = PhotoImage(file = filename)
08      pLabel.configure(image = photo)
09      pLabel.image = photo
10
11  def func_exit() :
12      window.quit()
13      window.destroy()
14
15  ## 메인 코드 부분 ##
16  window = Tk()
```

04 메뉴와 대화상자

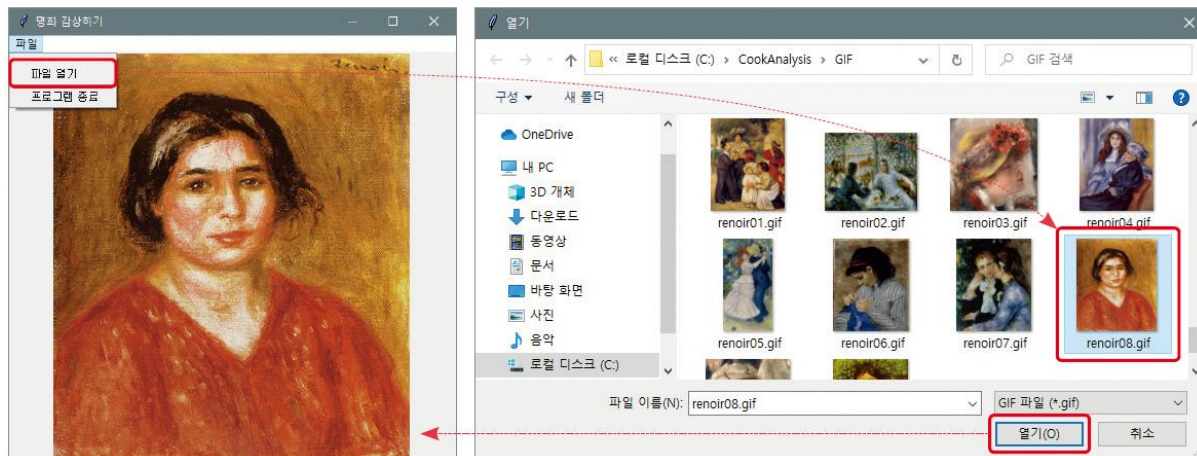
■ [프로그램 1] 완성

Code01-16.py

```
17 window.geometry("500x500")
18 window.title("영화 감상하기")
19
20 photo = PhotoImage()
21 pLabel = Label(window, image = photo)
22 pLabel.pack(expand = 1, anchor = CENTER)
23
24 mainMenu = Menu(window)
25 window.config(menu = mainMenu)
26 fileMenu = Menu(mainMenu)
27 mainMenu.add_cascade(label = "파일", menu = fileMenu)
28 fileMenu.add_command(label = "파일 열기", command = func_open)
29 fileMenu.add_separator()
30 fileMenu.add_command(label = "프로그램 종료", command = func_exit)
31
32 window.mainloop()
```

04 메뉴와 대화상자

■ [프로그램 1] 완성



04 메뉴와 대화상자

■ [프로그램 1] 완성

SELF STUDY 5-2

Code05-16.py를 수정해서 이미지가 회색 영상(Greyscale Image)으로 보이도록 하자.

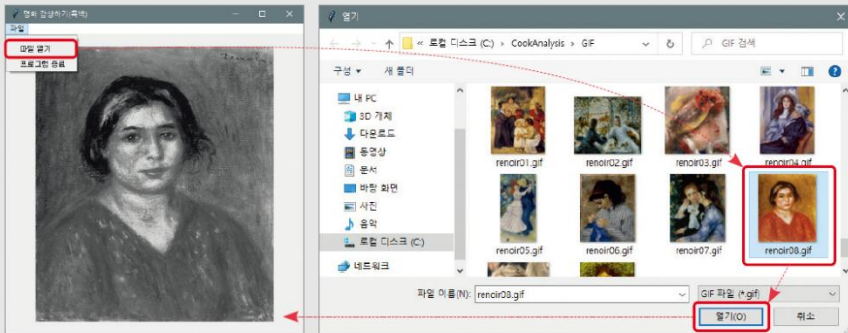
힌트1 이미지 크기는 photo.width()와 photo.height()로 알아낸다.

힌트2 각 픽셀의 색상값은 photo.get(x위치, y위치)로 알아낸다. 반환값은 (red, green, blue) 튜플이다.

힌트3 색상을 회색(Grey)값으로 하려면 (red+green+blue)/3으로 처리한다.

힌트4 이미지의 픽셀값을 변경하려면 다음 코드를 사용한다.

```
photo.put("#%02x"%02x"%02x"%02x" % (회색값, 회색값, 회색값), (x위치, y위치))
```



04 메뉴와 대화상자

■ [프로그램 1] 완성-회색처리

```
from tkinter import *
from tkinter.filedialog import *
from tkinter.simpledialog import *

## 함수 선언 부분 ##
def func_open() :
    filename = askopenfilename(parent = window, filetypes = (("GIF 파일", "*.gif"), ("모든 파일", "*.*")))
    photo = PhotoImage(file = filename)

    width = photo.width()
    height = photo.height()

    for i in range(height) :
        for k in range(width) :
            r, g, b = photo.get(k, i)
            grey = int((r+g+b)/3)
            photo.put("#%02x%02x%02x" % (grey, grey, grey), (k, i))

    pLabel.configure(image = photo)
    pLabel.image = photo

def func_exit() :
    window.quit()
    window.destroy()
```

04 메뉴와 대화상자

■ [프로그램 1] 완성-회색처리

메인 코드 부분

```
window = Tk()
```

```
window.geometry("500x500")
```

```
window.title("영화 감상하기(흑백)")
```

```
photo = PhotoImage()
```

```
pLabel = Label(window, image = photo)
```

```
pLabel.pack(expand=1, anchor = CENTER)
```

```
mainMenu = Menu(window)
```

```
window.config(menu = mainMenu)
```

```
fileMenu = Menu(mainMenu)
```

```
mainMenu.add_cascade(label = "파일", menu = fileMenu)
```

```
fileMenu.add_command(label = "파일 열기", command = func_open)
```

```
fileMenu.add_separator()
```

```
fileMenu.add_command(label = "프로그램 종료", command = func_exit)
```

05 파일 입출력

■ 파일 입출력의 개념

- 지금까지 배운 콘솔(Console)의 표준 입출력과 개념이 비슷함

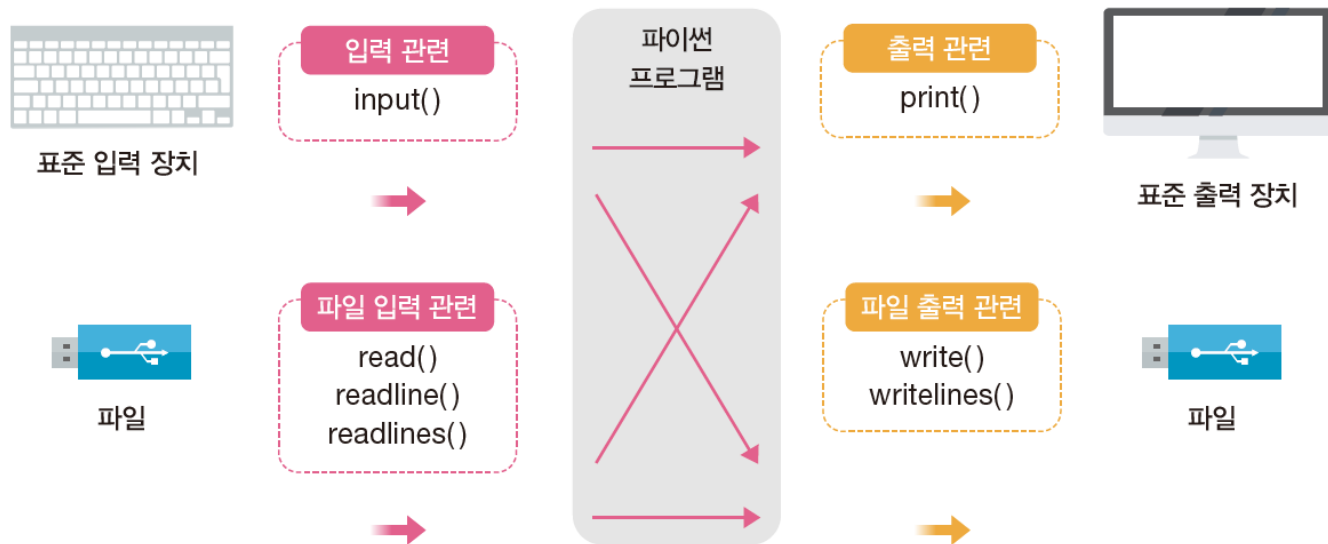


그림 5-2 표준 입출력과 파일 입출력 함수

05 파일 입출력

■ 파일 입출력 개념

- 키보드와 화면의 입출력 담당: `input( )`, `print( )`
- 파일에서 내용을 읽어옴: `read( )`, `readline( )`, `readlines( )`
- 파일에 내용을 씀: `write( )`, `writelines( )`
- 필요하다면 키보드에서 입력받은 후 파일에 쓸 수도 있고, 파일에서 내용을 입력받은 후 화면에 출력할 수도 있음

■ 파일 입출력 과정

- 표준 입출력과 달리 추가로 필요한 두 가지 작업:

파일을 사용하기 전의 '파일 열기' 작업과 파일 사용이 끝난 후의 '파일 닫기' 작업



그림 5-3 파일 처리의 3단계

■ 1단계 : 파일 열기

- 파일을 열려면 `open()` 함수에서 파일명을 입력하고, 읽기 또는 쓰기인지 지정

읽기용 : 변수명 = `open("파일명", "r")`

쓰기용 : 변수명 = `open("파일명", "w")`

- 읽기용 파일과 쓰기용 파일의 작업이 거의 동일
- 차이점은 `open()` 함수의 마지막 매개변수로, 읽기용은 `r`을 사용하고, 쓰기용은 `w`를 사용

■ 1단계 : 파일 열기

- open() 함수의 마지막 매개변수를 모드(mode)라고 함
- 파일을 열 때 어떤 용도로 여는지 결정해 주는 것
- r과 w뿐만 아니라 다양한 파일 열기 모드가 있음

표 5-2 파일의 열기 모드

종류	설명
생략	r과 동일하다.
r	읽기 모드. 기본값이다.
w	쓰기 모드. 기존에 파일이 있으면 덮어쓴다.
r+	읽기/쓰기 겸용 모드.
a	쓰기 모드. 기존에 파일이 있으면 이어서 쓴다. append의 약어이다.
t	텍스트 모드. 텍스트 파일을 처리한다. 기본값이다.
b	이진 모드. 이진 파일을 처리한다.

■ 2단계 : 파일 처리

- 파일에 데이터를 쓰거나 파일에서 데이터를 읽어 올 수 있는 상태가 되었음
- 파일을 처리하는 세부적인 방법은 예제를 통해 다루도록 함

05 파일 입출력

■ 3단계 : 파일 닫기

- 파일과 관련된 모든 작업이 끝나면 파일을 정상적으로 닫아야 함
- `open( )` 함수로 연 변수명을 이용하여 파일을 닫음

```
변수명.close();
```

05 파일 입출력

■ 텍스트 파일을 이용한 입력

- 데이터가 많고 복잡하다면 키보드를 통한 입력보다는 파일을 통해 입력 받는 것이 효율적



그림 5-4 파일 입력과 표준 출력

■ 한 행씩 읽어 들이기

- 먼저 메모장을 실행해서 적당한 내용을 세 줄 정도 입력한 후 저장

CookBook 데이터 분석을 공부합니다.
완전 재미있어요. ^^
데이터 분석을 공부하기 잘했네요~~

- C:\Temp 폴더에 data1.txt라는 이름으로 저장
- C 드라이브에 Temp 폴더가 없다면 미리 만들어 놓아야 함

05 파일 입출력

■ 한 행씩 읽어 들이기

Code01-17.py

```
01  inFp = None # 입력 파일
02  inStr = "" # 읽어 온 문자열
03
04  inFp = open("C:/Temp/data1.txt", "r")
05
06  inStr = inFp.readline()
07  print(inStr, end = "")
08
09  inStr = inFp.readline()
10  print(inStr, end = "")
11
12  inStr = inFp.readline()
13  print(inStr, end = "")
14
15  inFp.close()
```

실행 결과

CookBook 데이터 분석을 공부합니다.
완전 재미있어요. ^^
데이터 분석을 공부하기 잘했네요~~

05 파일 입출력

■ 한 행씩 읽어 들이기

- 파일에 있는 모든 행을 읽기 위해 수정된 코드

Code01-18.py

```
01  inFp = None # 입력 파일
02  inStr = "" # 읽어 온 문자열
03
04  inFp = open("C:/Temp/data1.txt", "r")
05
06  while True :
07      inStr = inFp.readline()
08      if inStr == "" :
09          break;
10      print(inStr, end = "")
11
12  inFp.close()
```

05 파일 입출력

■ 한 행씩 읽어 들이기

SELF STUDY 5-3

Code05-18.py를 수정해서 다음과 같이 앞에 각 행 번호를 출력해 보자.

힌트 1, 2, 3, ...으로 변경되는 변수를 하나 추가하고, print() 함수에서 "%d %s"를 사용한다.

실행 결과

```
1 : CookBook 데이터 분석을 공부합니다.  
2 : 완전 재미있어요. ^^  
3 : 데이터 분석을 공부하기 좋았네요~~
```

05 파일 입출력

■ 한 번에 모두 읽어 들이기

- `readline()` 함수는 한 행씩 읽지만, `readlines()` 함수는 파일 내용을 통째로 읽어서 리스트에 저장함

Code01-19.py

```
01 inFp = None
02 inList = ""
03
04 inFp = open("C:/Temp/data1.txt", "r")
05
06 inList = inFp.readlines()
07 print(inList)
08
09 inFp.close()
```

실행 결과

```
['CookBook 데이터 분석을 공부합니다.\n', '완전 재미있어요. ^^\\n', '데이터 분석을 공부하기 잘했네  
요~~']
```

■ 한 번에 모두 읽어 들이기

- Code01-19.py에서 파일의 내용을 한 행씩 출력하려면 4장에서 배운 리스트 처리 방식을 이용하면 됨

Code01-20.py

```
01 inFp = None
02 inList, inStr = [], ""
03
04 inFp = open("C:/Temp/data1.txt", "r")
05
06 inList = inFp.readlines()
07 for inStr in inList :
08     print(inStr, end = "")
09
10 inFp.close()
```

실행 결과

CookBook 데이터 분석을 공부합니다.
완전 재미있어요. ^^
데이터 분석을 공부하기 잘했네요~~

05 파일 입출력

■ 한 번에 모두 읽어 들이기

SELF STUDY 5-4

Code05-20.py를 수정해서 다음과 같이 앞에 각 행 번호를 출력해 보자.

실행 결과

- 1 : CookBook 데이터 분석을 공부합니다.
- 2 : 완전 재미있어요. ^^
- 3 : 데이터 분석을 공부하기 잘했네요~~

05 파일 입출력

■ 텍스트 파일을 이용한 출력

- 파일에 내용을 쓸 때는 `write()`, `writelines()` 함수를 사용하면 파일에 영구적으로 저장됨

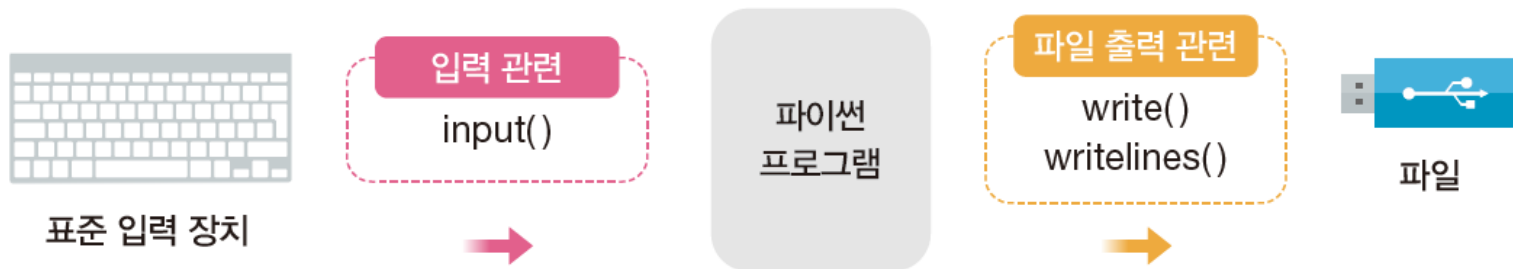


그림 5-5 표준 입력과 파일 출력

05 파일 입출력

■ 한 행씩 파일에 쓰기

- 키보드에서 입력한 내용을 한 행씩 파일에 쓰는 코드
- 아무것도 입력하지 않은 채 Enter 를 누르면 파일 쓰기를 종료

Code01-21.py

```
01 outFp = None
02 outStr = ""
03
04 outFp = open("C:/Temp/data2.txt", "w" )
05
06 while True :
07     outStr = input("내용 입력 : ")
08     if outStr != "" :
09         outFp.writelines(outStr + "\n")
10     else :
11         break
12
13 outFp.close()
14 print("--- 정상적으로 파일에 씀 ---")
```

실행 결과

내용 입력 : 데이터 분석을

내용 입력 : 열공하고

내용 입력 : 있습니다. ^^

내용 입력 :

--- 정상적으로 파일에 씀 ---

05 파일 입출력

■ 한 행씩 파일에 쓰기

- 도스 명령어 중 하나인 copy 명령어(또는 유닉스의 cp 명령어)를 수행하는 프로그램
- copy 명령어는 주어진 파일을 복사해 똑같은 파일을 하나 더 생성함

```
copy 소스파일 타깃파일
```


05 파일 입출력

■ 한 행씩 파일에 쓰기

- [시작]-[모든 프로그램]-[보조 프로그램]-[명령 프롬프트] 메뉴를 선택하거나 Windows + R 을 누른 후 cmd 명령어를 입력해 명령 프롬프트창 열기
- 명령어를 입력하면 win.ini 파일이 data3.txt 파일로 복사됨

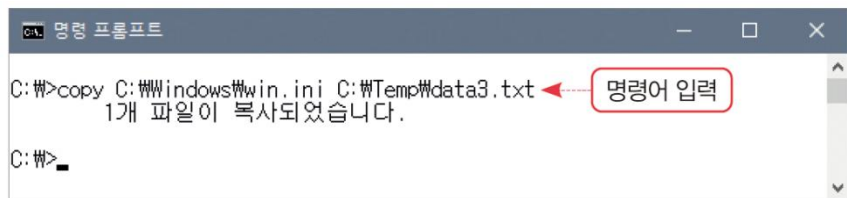


그림 5-6 copy 명령어 입력

05 파일 입출력

■ 한 행씩 파일에 쓰기

- copy 명령어의 구현 방식
 - 소스 파일은 읽기 모드로 파일을 열고, 타겟 파일은 쓰기 모드로 파일 열기
 - `readline()` 함수로 소스 파일의 내용을 읽어 와 `writelines()` 함수로 타겟 파일에 쓰기
 - 위의 과정을 파일의 끝을 만날 때까지 반복하기



그림 5-7 copy 명령어를 구현하는 파일 입출력

■ 한 행씩 파일에 쓰기

Code01-22.py

```
01  inFp, outFp = None, None
02  inStr = ""
03
04  inFp = open("C:/Windows/win.ini", "r")
05  outFp = open("C:/Temp/data3.txt", "w")
06
07  inList = inFp.readlines()
08  for inStr in inList :
09      outFp.writelines(inStr)
10
11  inFp.close()
12  outFp.close()
13  print("--- 파일이 정상적으로 복사되었음 ---")
```

실행 결과

--- 파일이 정상적으로 복사되었음 ---

06 이진 파일 입출력

■ 이진 파일의 개념

- 이진(Binary) 파일은 글자가 아닌 비트(Bit) 단위로 의미가 있는 파일
- 그림 파일, 음악 파일, 동영상 파일, 엑셀 파일, 실행 EXE 파일 등이 모두 이진 파일
- 텍스트 파일과 이진 파일을 구분하는 간단한 방법은 파일을 메모장에서 열어 보는 것



그림 5-8 텍스트 파일과 이진 파일을 메모장에서 열 때

06 이진 파일 입출력

■ 이진 파일 복사

- 이진 파일을 복사해서 새로운 이진 파일로 만드는 코드

Code01-23.py

```
01 inFp, outFp = None, None
02 inStr = ""
03
04 inFp = open("C:/Windows/write.exe", "rb")
05 outFp = open("C:/Temp/write.exe", "wb")
06
07 while True :
08     inStr = inFp.read(1)
09     if not inStr :
10         break
11     outFp.write(inStr)
12
13 inFp.close()
14 outFp.close()
15 print("--- 이진 파일이 정상적으로 복사되었음 ---")
```

실행 결과

--- 이진 파일이 정상적으로 복사되었음 ---

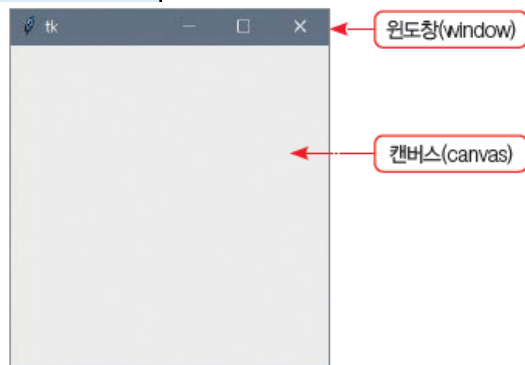
06 이진 파일 입출력

■ [프로그램 2] 완성

- 윈도우창 작성
 - 기본적인 윈도우창을 표시하는 간단한 코드

Code01-24.py

```
01 from tkinter import *
02
03 ## 변수 선언 부분 ##
04
05 window = None
06 canvas = None
07 XSIZE, YSIZE = 256, 256
08
09 ## 메인 코드 부분 ##
10 window = Tk()
11 canvas = Canvas(window, height = XSIZE, width = YSIZE)
12
13 canvas.pack()
14 window.mainloop()
```



06 이진 파일 입출력

■ [프로그램 2] 완성

■ 윈도우창 작성

- Code01-24.py의 11행에 다음 두 행을 추가로 코딩

```
paper = PhotoImage(width = XSIZE, height = YSIZE)  
canvas.create_image((XSIZE / 2, YSIZE / 2), image = paper, state = "normal")
```

- 잠시 후에 이 종이(paper)에 점을 256×256개 찍어서 사진을 화면에 출력할 것

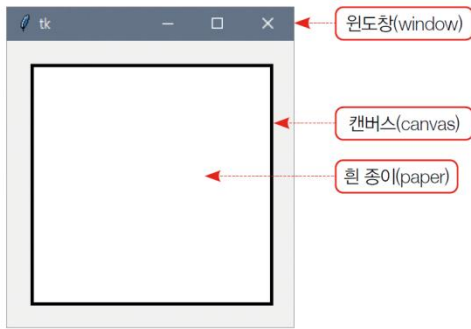


그림 5-10 흰 종이를 붙인 개념의 윈도우창

06 이진 파일 입출력

■ [프로그램 2] 완성

- RAW 사진 파일의 화면 출력

RAW 파일(디스크에 저장)



①
loadImage()
함수



메모리(2차원 리스트 : inImage)

25	77	88	33																
99	10	55	34																
98	32	11	78																

256×256 크기의 리스트
0~255 숫자로 채워짐

②
displayImage()
함수



모니터 화면에 출력

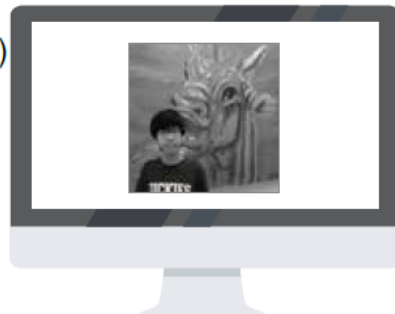


그림 5-11 RAW 사진 파일을 메모리로 읽은 후 화면에 출력하는 개념

06 이진 파일 입출력

■ [프로그램 2] 완성

■ RAW 사진 파일의 화면 출력

① 디스크에 저장된 RAW 파일을 메모리(inImage)로 불러들임

loadImage() 함수에 작성

```
1  for i in range(0, XSIZE) :  
2      tmpList = []  
3      for k in range(0, YSIZE) :  
4          data = int(ord(fp.read(1)))  
5          tmpList.append(data)  
6      inImage.append(tmpList)
```

06 이진 파일 입출력

■ [프로그램 2] 완성

■ RAW 사진 파일의 화면 출력

② 메모리(inImage)를 윈도우창에 출력하는 기능은 displayImage() 함수에 구현

```
1 rgbString = ""
2 for i in range(0, XSIZE) :
3     tmpString = ""
4     for k in range(0, YSIZE) :
5         data = image[i][k]
6         tmpString += "%02x%02x%02x " % (data, data, data) # x 뒤에 한 칸 공백
7     rgbString += "{" + tmpString + "}" # } 뒤에 한 칸 공백
8 paper.put(rgbString)
```

06 이진 파일 입출력

■ [프로그램 2] 완성

Code01-25.py

```
01  from tkinter import *
02
03  ## 함수 선언 부분 ##
04  def loadImage(fname) :
05      global inImage, XSIZE, YSIZE
06      fp = open(fname, 'rb')
07
08      for i in range(0, XSIZE) :
09          tmpList = []
10          for k in range(0, YSIZE) :
11              data = int(ord(fp.read(1)))
12              tmpList.append(data)
13              inImage.append(tmpList)
14
15      fp.close()
16
```

06 이진 파일 입출력

■ [프로그램 2] 완성

Code01-25.py

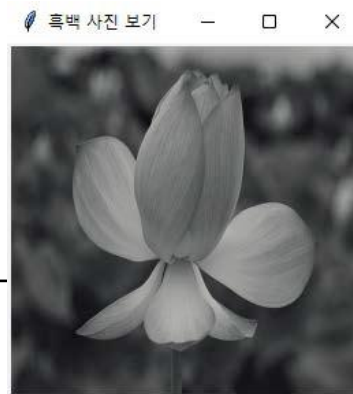
```
17 def displayImage(image) :
18     global XSIZE, YSIZE
19     rgbString = ""
20     for i in range(0, XSIZE) :
21         tmpString = ""
22         for k in range(0, YSIZE) :
23             data = image[i][k]
24             tmpString += "%02x%02x%02x " % (data, data, data) # x 뒤에 한 칸 공백
25             rgbString += "{" + tmpString + "}" # } 뒤에 한 칸 공백
26         paper.put(rgbString)
27
28 ## 전역 변수 선언 부분 ##
29 window = None
30 canvas = None
31 XSIZE, YSIZE = 256, 256
32 inImage = [] # 2차원 리스트(메모리)
33
34 ## 메인 코드 부분 ##
```

06 이진 파일 입출력

■ [프로그램 2] 완성

Code01-25.py

```
35 window = Tk()
36 window.title("흑백 사진 보기")
37 canvas = Canvas(window, height = XSIZE, width = YSIZE)
38 paper = PhotoImage(width = XSIZE, height = YSIZE)
39 canvas.create_image(XSIZE / 2, YSIZE / 2, image = paper, state = "normal")
40
41 # 파일 --> 메모리
42 filename = 'RAW/tree.raw' # C:/CookAnalysis/RAW/tree.raw
43 loadImage(filename)
44
45 # 메모리 --> 화면
46 displayImage(inImage)
47
48 canvas.pack()
49 window.mainloop()
```



Thank You
